COMPLETE REFERENCE GUIDE

CSS Mastery

From Absolute Basics to Modern Advanced CSS
— Every Topic, Every Module, Every Example

15 MODULES · 100+ TOPICS · 200+ CODE EXAMPLES

---

title: "Complete CSS Mastery Guide" subtitle: "From Basics to Advanced — Modules, Topics & Examples" author: "Your CSS Coach" date: "" toc: true toc-depth: 3 numbersections: true geometry: margin=1in fontsize: 10pt colorlinks: true

---

How to Use This Guide

This document is organized into **15 Modules**, moving progressively from absolute basics to advanced, modern CSS (2025-2026 era features). Every topic includes:

- A clear explanation of **what it is** and **why it matters**
- The **syntax**
- A **working code example**
- Notes on **common mistakes / best practices** where relevant

Treat this as a single reference. Read modules in order if you're learning from scratch, or jump to any module/topic if you're filling a gap.

MODULE 1: CSS Fundamentals

1.1 What is CSS?

CSS (Cascading Style Sheets) is the language used to describe the **presentation** of an HTML document — colors, layout, fonts, spacing, animations, and more. HTML provides structure/content; CSS provides style.

- **Cascading**: multiple rules can apply to the same element, and CSS has rules to decide which one "wins".
- CSS works by **selecting** HTML elements and applying **declarations** (property-value pairs) to them.

```
/* selector { property: value; } */
p {
  color: navy;
  font-size: 16px;
}
```

1.2 Ways to Add CSS to HTML

There are three ways to apply CSS:

1. Inline CSS — written directly on an element using the `style` attribute. Highest specificity, hardest to maintain.

```
<p style="color: red; font-weight: bold;">This is inline styled text.</p>
```

2. Internal (Embedded) CSS — written inside a `<style>` tag in the `<head>` of the HTML document.

```
<head>
  <style>
    p {
      color: green;
      font-size: 18px;
    }
  </style>
</head>
```

3. External CSS — written in a separate `.css` file and linked via `<link>`. This is the **recommended** approach for real projects (cacheable, reusable, separates concerns).

```
<head>
  <link rel="stylesheet" href="styles.css">
</head>
```

```
/* styles.css */
body {
  font-family: Arial, sans-serif;
  margin: 0;
}
```

1.3 CSS Syntax Anatomy

```
selector {
  property: value;     /* declaration */
  property2: value2;
}
```

- **Selector** — targets which element(s) to style.
- **Declaration block** — wrapped in `{ }`, contains one or more declarations.
- **Declaration** — a `property: value;` pair, ending with a semicolon.

1.4 Comments in CSS

Comments are written using `/* ... */` and are ignored by the browser. They can span multiple lines.

```
/* This is a single-line comment */

/*
  This is a
  multi-line comment
*/

p {
  color: blue; /* inline comment explaining this rule */
}
```

> **Best Practice:** Use comments to explain *why* something is done (e.g., a hack for a browser bug), not *what* the obvious code does.

1.5 Basic Selectors

Selector	Example	Matches
Universal	`* { }`	All elements
Type/Element	`p { }`	All `<p>` elements
Class	`.box { }`	All elements with `class="box"`
ID	`#header { }`	The element with `id="header"`
Grouping	`h1, h2, h3 { }`	All listed elements

```css
* {
  box-sizing: border-box;
}

p {
  line-height: 1.6;
}

.highlight {
  background-color: yellow;
}

#main-title {
  font-size: 2.5rem;
}

h1, h2, h3 {
  font-family: "Georgia", serif;
}
```

```html
<h1 id="main-title">Welcome</h1>
<p class="highlight">This paragraph is highlighted.</p>
```

1.6 Combinators

Combinators define relationships between selectors.

```css
/* Descendant combinator: any <span> inside .card, at any depth */
.card span {
  color: gray;
}

/* Child combinator: only direct children <li> of <ul> */
ul > li {
  list-style: square;
}

/* Adjacent sibling: the <p> immediately after an <h2> */
h2 + p {
  font-weight: bold;
}

/* General sibling: all <p> elements that are siblings after <h2> */
h2 ~ p {
  color: #555;
}
```

```html
<div class="card">
  <h3>Title <span>(new)</span></h3>
</div>
```

1.7 The Cascade, Specificity & Inheritance

Cascade is the algorithm browsers use to decide which CSS rule applies when multiple rules target the same element. Order of priority (highest wins):

1. `!important` declarations
2. Inline styles
3. Specificity of the selector

4. Source order (later rules override earlier ones if specificity is equal)

Specificity calculation — each selector gets a score `(a, b, c, d)`:

- `a` = inline styles (1 if present)
- `b` = number of ID selectors
- `c` = number of class, attribute, and pseudo-class selectors
- `d` = number of element/type selectors and pseudo-elements

```
/* specificity (0,0,0,1) */
p { color: blue; }

/* specificity (0,0,1,0) - wins over above */
.text { color: green; }

/* specificity (0,1,0,0) - wins over above */
#unique { color: red; }

/* specificity (0,1,1,1) - highest among these */
#unique.text p { color: purple; }
```

```
/* !important overrides normal specificity rules (use sparingly) */
p {
  color: black !important;
}
```

Inheritance — some properties (like `color`, `font-family`, `line-height`) are inherited by child elements automatically; others (like `border`, `margin`, `padding`) are not.

```
body {
  color: #333;       /* inherited by all text inside body */
  font-family: Arial;
}

/* Force inheritance explicitly */
.child {
  border-color: inherit;
}
```

MODULE 2: Colors, Units & Values

2.1 Color Formats

CSS supports multiple ways to define color:

```
.color-examples {
  /* Named colors */
  color: tomato;

  /* Hexadecimal */
  background-color: #1e90ff;      /* 6-digit */
  border-color: #f0f8;            /* 4-digit with alpha (shorthand) */

  /* RGB / RGBA */
  outline-color: rgb(34, 139, 34);
  box-shadow: 0 0 10px rgba(0, 0, 0, 0.5);

  /* HSL / HSLA (Hue, Saturation, Lightness, Alpha) */
  color: hsl(210, 100%, 56%);
  background: hsla(210, 100%, 56%, 0.3);

  /* Modern: hex with alpha, rgb()/hsl() without commas */
  border: 1px solid rgb(0 0 0 / 40%);
}

/* currentColor: reuses the element's text color elsewhere */
.icon {
  color: crimson;
  border: 2px solid currentColor; /* border becomes crimson */
}
```

> **Tip:** HSL is great for design systems because you can adjust lightness/saturation to create tints and shades while keeping the same hue.

2.2 Units of Measurement

Absolute Units

```
.absolute-units {
  width: 200px;    /* pixels - most common absolute unit */
  height: 5cm;     /* centimeters - rarely used on screen */
  margin: 1in;     /* inches */
}
```

Relative Units

```
html {
  font-size: 16px; /* default browser size, used as base for rem */
}

.relative-units {
  font-size: 1.5em;   /* relative to PARENT element's font-size */
  font-size: 1.5rem;  /* relative to ROOT (html) font-size — predictable! */
  width: 50%;         /* relative to parent's width */
  width: 100vw;       /* 100% of viewport width */
  height: 100vh;      /* 100% of viewport height */
  padding: 5vmin;     /* relative to smaller of vw/vh */
  font-size: 4vmax;   /* relative to larger of vw/vh */
  width: 40ch;        /* relative to width of the "0" character — great for text columns */
}
```

rem vs em: `rem` is always relative to the root `<html>` font-size (predictable, recommended for most sizing). `em` is relative to the *parent's* font-size, which can compound and become unpredictable in nested elements.

2.3 CSS Value Functions

```
.value-functions {
  /* calc(): mix units and do math */
  width: calc(100% - 60px);
  height: calc(100vh - 4rem);

  /* min(), max(): pick smallest/largest value */
  width: min(90%, 600px);     /* never wider than 600px, but shrinks on small screens */
  font-size: max(1rem, 2vw);

  /* clamp(min, preferred, max): responsive value in one line */
  font-size: clamp(1rem, 2.5vw, 2rem);
  padding: clamp(1rem, 5%, 3rem);
}
```

2.4 The `box-sizing` Property (Preview)

```
/* content-box (default): width/height apply ONLY to content,
   padding & border are ADDED on top */
.box-default {
  box-sizing: content-box;
  width: 200px;
  padding: 20px;
  border: 5px solid black;
  /* Rendered width = 200 + 40 + 10 = 250px */
}

/* border-box: width/height INCLUDE padding & border — predictable sizing */
.box-better {
  box-sizing: border-box;
  width: 200px;
  padding: 20px;
  border: 5px solid black;
  /* Rendered width = exactly 200px */
}

/* Best practice: apply to everything */
*, *::before, *::after {
  box-sizing: border-box;
}
```

MODULE 3: The Box Model & Display

3.1 The CSS Box Model

Every HTML element is a rectangular box made of four parts, from inside out:

1. **Content** — the actual text/image
2. **Padding** — space between content and border (inside the element's background)
3. **Border** — the edge around the padding
4. **Margin** — space outside the border, separating elements

```
.box {
  width: 300px;
  height: 150px;
  padding: 20px;
  border: 4px solid #333;
  margin: 30px;
  background-color: lightblue;
}
```

Shorthand vs Longhand

```
.spacing-shorthand {
  /* shorthand: top right bottom left (clockwise) */
  margin: 10px 20px 30px 40px;
  padding: 10px 20px; /* top/bottom = 10px, left/right = 20px */
  margin: 15px;       /* all four sides = 15px */
}

.spacing-longhand {
  margin-top: 10px;
  margin-right: 20px;
  margin-bottom: 30px;
  margin-left: 40px;
}
```

Margin Collapsing

Vertical margins between adjacent block elements "collapse" into a single margin (the larger of the two), not added together.

```
.box-a { margin-bottom: 20px; }
.box-b { margin-top: 30px; }
/* The actual gap between box-a and box-b is 30px, not 50px */
```

3.2 The `display` Property

```css
/* block: takes full width, starts on a new line (div, p, h1...) */
.block {
  display: block;
}

/* inline: only takes as much width as content, no line break (span, a, strong...) */
.inline {
  display: inline;
  /* width, height, margin-top/bottom have NO effect on inline elements */
}

/* inline-block: flows inline BUT respects width/height/margin like block */
.inline-block {
  display: inline-block;
  width: 120px;
  height: 40px;
  margin: 0 10px;
}

/* none: removes element entirely from layout (no space reserved) */
.hidden {
  display: none;
}

/* flex & grid: covered in dedicated modules */
.flex-container { display: flex; }
.grid-container { display: grid; }
```

```html
<div class="block">I'm a block (full width)</div>
<span class="inline">I'm inline</span>
<span class="inline">Another inline</span>
<div class="inline-block">inline-block A</div>
<div class="inline-block">inline-block B</div>
```

3.3 `visibility` vs `display: none`

```css
/* display: none — element is removed from layout, takes no space */
.gone {
  display: none;
}

/* visibility: hidden — element is invisible but STILL occupies space */
.invisible {
  visibility: hidden;
}

/* visibility: collapse — mainly for table rows/columns */
.collapsed-row {
  visibility: collapse;
}
```

3.4 `overflow` Property

Controls what happens when content is too big for its container.

```
.overflow-visible { overflow: visible; }  /* default: content spills out */
.overflow-hidden  { overflow: hidden;  }  /* clips extra content */
.overflow-scroll  { overflow: scroll;  }  /* always shows scrollbars */
.overflow-auto    { overflow: auto;    }  /* scrollbars only when needed */

/* Separate axes */
.overflow-mixed {
  overflow-x: hidden;
  overflow-y: auto;
}
```

```
<div style="width:200px; height:100px; overflow:auto; border:1px solid #ccc;">
  This is a long block of text that will overflow the fixed-size container
  and trigger a scrollbar because overflow is set to auto...
</div>
```

MODULE 4: Backgrounds & Borders

4.1 Background Properties

```
.bg-color {
  background-color: #f4f4f4;
}

.bg-image {
  background-image: url("hero.jpg");
  background-repeat: no-repeat;     /* repeat | repeat-x | repeat-y | no-repeat */
  background-position: center top;  /* keyword, %, or px */
  background-size: cover;           /* cover | contain | <width> <height> */
  background-attachment: fixed;     /* scroll | fixed | local */
}

/* Shorthand */
.bg-shorthand {
  background: #222 url("pattern.png") no-repeat center / cover;
}

/* Multiple backgrounds (layered) */
.bg-layers {
  background:
    linear-gradient(rgba(0,0,0,0.4), rgba(0,0,0,0.4)),
    url("photo.jpg") center / cover no-repeat;
}
```

4.2 Gradients

```
.linear-gradient {
  background: linear-gradient(to right, #ff7e5f, #feb47b);
}

.linear-gradient-angle {
  background: linear-gradient(45deg, #36d1dc, #5b86e5);
}

.radial-gradient {
  background: radial-gradient(circle, #fdfbfb, #ebedee);
}

.conic-gradient {
  background: conic-gradient(from 90deg, red, yellow, green, blue, red);
}

/* Multiple color stops */
.multi-stop {
  background: linear-gradient(
    to right,
    red 0%,
    yellow 25%,
    green 50%,
    blue 100%
  );
}
```

4.3 Border Properties

```
.border-basic {
  border-width: 2px;
  border-style: solid;   /* solid | dashed | dotted | double | groove | ridge | inset | outset | none *
  border-color: #333;
}

/* Shorthand */
.border-shorthand {
  border: 2px solid #333;
}

/* Individual sides */
.border-sides {
  border-top: 4px solid red;
  border-right: 1px dashed blue;
  border-bottom: 2px dotted green;
  border-left: none;
}
```

4.4 `border-radius`

```
.rounded {
  border-radius: 8px; /* all corners */
}

.pill-button {
  border-radius: 9999px; /* fully rounded ends */
}

.circle {
  width: 100px;
  height: 100px;
  border-radius: 50%; /* perfect circle if width = height */
}

/* Individual corners: top-left, top-right, bottom-right, bottom-left */
.custom-corners {
  border-radius: 20px 5px 20px 5px;
}

/* Elliptical corners: horizontal-radius / vertical-radius */
.elliptical {
  border-radius: 50% / 20%;
}
```

4.5 `box-shadow` and `outline`

```
/* offset-x | offset-y | blur-radius | spread-radius | color */
.shadow-basic {
  box-shadow: 2px 4px 10px rgba(0, 0, 0, 0.25);
}

/* Inset shadow */
.shadow-inset {
  box-shadow: inset 0 2px 4px rgba(0, 0, 0, 0.3);
}

/* Multiple shadows (layered) */
.shadow-layered {
  box-shadow:
    0 1px 2px rgba(0,0,0,0.2),
    0 4px 12px rgba(0,0,0,0.15);
}

/* Outline: drawn OUTSIDE the border, doesn't affect layout box.
   Great for accessibility focus states */
.focus-ring:focus {
  outline: 3px solid #4d90fe;
  outline-offset: 2px;
}
```

MODULE 5: Typography

5.1 Font Properties

```
.font-basics {
  font-family: "Helvetica Neue", Arial, sans-serif; /* fallback stack */
  font-size: 18px;
  font-weight: 700;          /* 100-900, or normal/bold */
  font-style: italic;        /* normal | italic | oblique */
  font-variant: small-caps;
  line-height: 1.6;          /* unitless = multiplier of font-size (recommended) */
}

/* Shorthand: font-style font-weight font-size/line-height font-family */
.font-shorthand {
  font: italic 700 18px/1.6 Arial, sans-serif;
}
```

5.2 Text Properties

```
.text-properties {
  color: #222;
  text-align: center;          /* left | right | center | justify */
  text-decoration: underline;  /* none | underline | line-through | overline */
  text-decoration: underline dotted red; /* line + style + color */
  text-transform: uppercase;   /* uppercase | lowercase | capitalize | none */
  text-indent: 2em;
  letter-spacing: 0.05em;
  word-spacing: 0.2em;
  white-space: nowrap;         /* normal | nowrap | pre | pre-wrap | pre-line */
  text-overflow: ellipsis;     /* requires overflow:hidden + white-space:nowrap */
  overflow: hidden;
  word-break: break-word;      /* prevents long words from overflowing */
}
```

```
<p class="text-properties" style="width:150px;">
  This is a very long sentence that will be truncated with an ellipsis.
</p>
```

5.3 Web Fonts with `@font-face`

```
@font-face {
  font-family: "MyCustomFont";
  src: url("fonts/MyFont.woff2") format("woff2"),
       url("fonts/MyFont.woff") format("woff");
  font-weight: 400;
  font-style: normal;
  font-display: swap; /* avoids invisible text while font loads */
}

body {
  font-family: "MyCustomFont", sans-serif;
}
```

Using Google Fonts

```
<link rel="preconnect" href="https://fonts.googleapis.com">
<link href="https://fonts.googleapis.com/css2?family=Inter:wght@400;700&display=swap" rel="stylesheet">
```

```
body {
  font-family: "Inter", sans-serif;
}
```

5.4 Lists

```
ul.custom-list {
  list-style-type: square;     /* disc | circle | square | decimal | none */
  list-style-position: inside; /* inside | outside */
  list-style-image: url("bullet.png");
}

/* Shorthand */
ul.custom-list-shorthand {
  list-style: square inside url("bullet.png");
}

/* Removing default list styling (common for nav menus) */
nav ul {
  list-style: none;
  margin: 0;
  padding: 0;
}
```

MODULE 6: Positioning, Float & Stacking

6.1 The `position` Property

```
/* static (default): follows normal document flow.
   top/right/bottom/left have NO effect */
.static {
  position: static;
}

/* relative: stays in normal flow, but offsets are relative to its NORMAL position.
   Also establishes a positioning context for absolute children */
.relative {
  position: relative;
  top: 10px;
  left: 20px;
}

/* absolute: removed from normal flow, positioned relative to the
   nearest ancestor with position != static (or the viewport if none) */
.absolute {
  position: absolute;
  top: 0;
  right: 0;
}

/* fixed: removed from flow, positioned relative to the VIEWPORT.
   Stays in place when scrolling */
.fixed-header {
  position: fixed;
  top: 0;
  left: 0;
  width: 100%;
}

/* sticky: behaves like relative until a scroll threshold is crossed,
   then "sticks" like fixed within its parent */
.sticky-nav {
  position: sticky;
  top: 0;
}
```

Practical Example: Badge on a Card

```css
.card {
  position: relative;
  width: 250px;
  padding: 16px;
  border: 1px solid #ddd;
}

.card .badge {
  position: absolute;
  top: -10px;
  right: -10px;
  background: crimson;
  color: white;
  border-radius: 50%;
  width: 28px;
  height: 28px;
  display: flex;
  align-items: center;
  justify-content: center;
}
```

```html
<div class="card">
  <span class="badge">3</span>
  <h3>Notifications</h3>
</div>
```

6.2 Float and Clear (Legacy Layout)

`float` was historically used for layouts before Flexbox/Grid. Today it's mainly used for wrapping text around images.

```css
.float-image {
  float: left;
  margin-right: 16px;
  width: 120px;
}

/* clear: prevents an element from wrapping around floated elements */
.clear-fix {
  clear: both;
}

/* Classic "clearfix" hack for containers with floated children */
.clearfix::after {
  content: "";
  display: table;
  clear: both;
}
```

```html
<div class="clearfix">
  <img src="avatar.jpg" class="float-image" alt="Avatar">
  <p>This text will wrap around the floated image on its right side...</p>
</div>
```

> **Modern advice:** Prefer Flexbox or Grid for layout. Use `float` only for its original purpose (text wrapping around media).

6.3 `z-index` and Stacking Contexts

`z-index` controls which elements appear on top when they overlap. It only works on positioned elements (`position` other than `static`).

```
.layer-back {
  position: absolute;
  z-index: 1;
  background: lightblue;
}

.layer-front {
  position: absolute;
  z-index: 10; /* higher value = on top */
  background: salmon;
  top: 20px;
  left: 20px;
}
```

Stacking context: Certain properties (e.g., `position` + `z-index`, `opacity < 1`, `transform`, `filter`) create a *new stacking context*, meaning `z-index` values inside it are only compared against each other, not the whole page. This is a common source of "why isn't my z-index working?" bugs.

MODULE 7: Flexbox (Flexible Box Layout)

Flexbox is a one-dimensional layout model — it arranges items in a row or column, distributing space and aligning items.

7.1 Setting Up a Flex Container

```
.flex-container {
  display: flex; /* or inline-flex */
}
```

7.2 Main Axis Properties (on the container)

```
.flex-container {
  display: flex;

  /* direction of the main axis */
  flex-direction: row; /* row | row-reverse | column | column-reverse */

  /* how items wrap onto multiple lines */
  flex-wrap: wrap; /* nowrap | wrap | wrap-reverse */

  /* shorthand for flex-direction + flex-wrap */
  flex-flow: row wrap;

  /* alignment along the MAIN axis */
  justify-content: space-between;
  /* options: flex-start | flex-end | center | space-between | space-around | space-evenly */

  /* alignment along the CROSS axis */
  align-items: center;
  /* options: stretch | flex-start | flex-end | center | baseline */

  /* alignment of wrapped LINES along the cross axis */
  align-content: space-between;

  gap: 16px; /* spacing between flex items (modern, recommended over margins) */
}
```

7.3 Flex Item Properties

```
.flex-item {
  /* how much an item GROWS relative to siblings when there's extra space */
  flex-grow: 1;

  /* how much an item SHRINKS relative to siblings when space is tight */
  flex-shrink: 1;

  /* the item's initial size before growing/shrinking */
  flex-basis: 200px;

  /* shorthand: grow shrink basis */
  flex: 1 1 200px;
  flex: 1; /* common shorthand meaning "grow to fill equally" */

  /* override align-items for THIS item only */
  align-self: flex-end;

  /* reorder items visually without changing HTML order */
  order: 2;
}
```

7.4 Practical Example 1: Navigation Bar

```
.navbar {
  display: flex;
  justify-content: space-between;
  align-items: center;
  padding: 1rem 2rem;
  background: #0f172a;
  color: white;
}

.navbar .nav-links {
  display: flex;
  gap: 1.5rem;
  list-style: none;
}
```

```
<nav class="navbar">
  <div class="logo">MyBrand</div>
  <ul class="nav-links">
    <li><a href="#">Home</a></li>
    <li><a href="#">About</a></li>
    <li><a href="#">Contact</a></li>
  </ul>
</nav>
```

7.5 Practical Example 2: Perfect Centering

```
.center-box {
  display: flex;
  justify-content: center; /* horizontal */
  align-items: center;     /* vertical */
  height: 100vh;
}
```

```
<div class="center-box">
  <div class="card">I'm perfectly centered!</div>
</div>
```

7.6 Practical Example 3: Equal-Height Responsive Cards

```
.card-row {
  display: flex;
  flex-wrap: wrap;
  gap: 1rem;
}

.card-row .card {
  flex: 1 1 250px; /* grow, shrink, base width 250px */
  background: #f1f5f9;
  padding: 1rem;
  border-radius: 8px;
}
```

```
<div class="card-row">
  <div class="card">Card 1</div>
  <div class="card">Card 2 with more content that makes it taller</div>
  <div class="card">Card 3</div>
</div>
```

MODULE 8: CSS Grid Layout

CSS Grid is a two-dimensional layout system — it controls rows AND columns simultaneously, making it ideal for full page layouts.

8.1 Setting Up a Grid Container

```
.grid-container {
  display: grid; /* or inline-grid */
  grid-template-columns: 200px 1fr 1fr; /* 3 columns: fixed, flexible, flexible */
  grid-template-rows: auto 1fr auto;    /* 3 rows */
  gap: 16px; /* shorthand for row-gap + column-gap */
}
```

8.2 The `fr` Unit and `repeat()`

```
.grid-equal-columns {
  display: grid;
  grid-template-columns: repeat(3, 1fr); /* 3 equal-width columns */
  gap: 1rem;
}

.grid-mixed {
  display: grid;
  /* fr distributes remaining space proportionally */
  grid-template-columns: 1fr 2fr 1fr; /* middle column is twice as wide */
}
```

8.3 `repeat()` with `auto-fit` / `auto-fill` — Responsive Grids Without Media Queries

```
.responsive-grid {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(200px, 1fr));
  gap: 1rem;
}
/* Creates as many 200px+ columns as fit, stretching to fill the row */
```

8.4 Placing Items: Lines, Spans & Areas

```
.grid-container {
  display: grid;
  grid-template-columns: repeat(4, 1fr);
  grid-template-rows: 80px 1fr 60px;
  gap: 10px;
}

.header {
  grid-column: 1 / -1;  /* span from first to last column line */
  grid-row: 1;
}

.sidebar {
  grid-column: 1 / 2;
  grid-row: 2;
}

.main {
  grid-column: 2 / 5;   /* spans columns 2,3,4 */
  grid-row: 2;
}

.footer {
  grid-column: 1 / -1;
  grid-row: 3;
}
```

Named Grid Areas (Most Readable Method)

```
.page-layout {
  display: grid;
  grid-template-columns: 200px 1fr;
  grid-template-rows: 70px 1fr 50px;
  grid-template-areas:
    "header header"
    "sidebar main"
    "footer footer";
  gap: 10px;
  min-height: 100vh;
}

.header  { grid-area: header; }
.sidebar { grid-area: sidebar; }
.main    { grid-area: main; }
.footer  { grid-area: footer; }
```

```
<div class="page-layout">
  <header class="header">Header</header>
  <aside class="sidebar">Sidebar</aside>
  <main class="main">Main Content</main>
  <footer class="footer">Footer</footer>
</div>
```

8.5 Aligning Items in Grid

```css
.grid-container {
  display: grid;
  /* alignment of items within their cells */
  justify-items: center; /* start | end | center | stretch (horizontal) */
  align-items: center;   /* start | end | center | stretch (vertical) */

  /* alignment of the WHOLE grid within the container */
  justify-content: center;
  align-content: center;
}

/* override for a single item */
.special-item {
  justify-self: end;
  align-self: start;
}
```

8.6 Implicit Grid & `grid-auto-rows` / `grid-auto-flow`

```css
.gallery {
  display: grid;
  grid-template-columns: repeat(4, 1fr);
  grid-auto-rows: 150px;   /* size of automatically-created rows */
  grid-auto-flow: dense;   /* fills gaps efficiently */
  gap: 8px;
}

.gallery .featured {
  grid-column: span 2;
  grid-row: span 2;
}
```

8.7 CSS Grid vs Flexbox — When to Use Which

Use Flexbox when...	Use Grid when...
Laying out items in a single row or column	Laying out a full 2D page structure
Content size should drive layout	You want explicit row & column control
Building navbars, button groups, toolbars	Building dashboards, galleries, page templates

MODULE 9: Responsive Web Design

9.1 The Viewport Meta Tag

Always include this in your HTML `<head>` — without it, mobile browsers render pages at desktop width and scale down.

```
<meta name="viewport" content="width=device-width, initial-scale=1.0">
```

9.2 Media Queries

Media queries apply CSS rules only when certain conditions (like screen width) are met.

```
/* Base styles (mobile-first approach) */
.container {
  padding: 1rem;
  font-size: 14px;
}

/* Tablet and up */
@media (min-width: 768px) {
  .container {
    padding: 2rem;
    font-size: 16px;
  }
}

/* Desktop and up */
@media (min-width: 1024px) {
  .container {
    padding: 3rem;
    max-width: 1200px;
    margin: 0 auto;
  }
}

/* Max-width queries (desktop-first approach) */
@media (max-width: 767px) {
  .sidebar {
    display: none;
  }
}

/* Combining conditions */
@media (min-width: 600px) and (max-width: 900px) {
  .grid { grid-template-columns: repeat(2, 1fr); }
}

/* Orientation */
@media (orientation: landscape) {
  .hero { height: 60vh; }
}

/* User preferences */
@media (prefers-color-scheme: dark) {
  body { background: #0f172a; color: #e2e8f0; }
}

@media (prefers-reduced-motion: reduce) {
  * { animation: none !important; transition: none !important; }
}
```

9.3 Mobile-First vs Desktop-First

Mobile-first (recommended): write base styles for small screens, then use `min-width` queries to enhance for larger screens.

```
/* Mobile (default) */
.nav { flex-direction: column; }

/* Tablet+ */
@media (min-width: 768px) {
  .nav { flex-direction: row; }
}
```

9.4 Responsive Typography & Layout

```
/* Fluid font-size without breakpoints */
h1 {
  font-size: clamp(1.75rem, 4vw + 1rem, 3.5rem);
}

/* Responsive grid that adapts automatically */
.cards {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(250px, 1fr));
  gap: 1rem;
}
```

9.5 Responsive Images

```
img {
  max-width: 100%;   /* never overflow its container */
  height: auto;      /* maintain aspect ratio */
  display: block;
}

.hero-img {
  width: 100%;
  aspect-ratio: 16 / 9;
  object-fit: cover; /* crop nicely without distortion */
}
```

```
<!-- srcset/sizes for serving different image resolutions -->
<img
  src="photo-800.jpg"
  srcset="photo-400.jpg 400w, photo-800.jpg 800w, photo-1200.jpg 1200w"
  sizes="(max-width: 600px) 100vw, 50vw"
  alt="Description"
>
```

9.6 Container Queries (Modern — Component-Level Responsiveness)

Unlike media queries (which respond to the *viewport*), container queries respond to the size of a *parent container* — perfect for reusable components.

```css
.card-container {
  container-type: inline-size;
  container-name: card;
}

.card {
  display: flex;
  flex-direction: column;
}

@container card (min-width: 400px) {
  .card {
    flex-direction: row;
  }
}
```

MODULE 10: Pseudo-classes & Pseudo-elements

10.1 What's the Difference?

- **Pseudo-classes** (single colon `:`) select elements based on **state** or **position** (e.g., hover, first-child).
- **Pseudo-elements** (double colon `::`) target a **part** of an element that doesn't exist in the HTML (e.g., `::before`).

10.2 Interactive Pseudo-classes

```
/* Mouse hover */
.button:hover {
  background-color: #2563eb;
  cursor: pointer;
}

/* Keyboard/click focus */
.input:focus {
  outline: 2px solid #2563eb;
  border-color: #2563eb;
}

/* Focus only when navigating via keyboard (not mouse click) */
.button:focus-visible {
  outline: 3px solid orange;
}

/* While being clicked/pressed */
.button:active {
  transform: scale(0.98);
}

/* Disabled form elements */
.input:disabled {
  background: #f1f1f1;
  cursor: not-allowed;
}

/* Checked checkbox/radio */
.checkbox:checked + label {
  font-weight: bold;
  color: green;
}

/* Form validation states */
.input:valid   { border-color: green; }
.input:invalid { border-color: red; }
.input:required { border-left: 3px solid orange; }
```

10.3 Structural / Positional Pseudo-classes

```
/* First and last child of a parent */
li:first-child { font-weight: bold; }
li:last-child  { border-bottom: none; }

/* nth-child(): powerful pattern matching */
tr:nth-child(odd)  { background: #f9f9f9; }   /* zebra striping */
tr:nth-child(even) { background: #ffffff; }
li:nth-child(3) { color: red; }               /* exactly the 3rd item */
li:nth-child(3n) { color: blue; }             /* every 3rd item: 3,6,9... */
li:nth-child(3n+1) { color: green; }          /* 1,4,7,10... */

/* nth-of-type: like nth-child but counts only matching siblings of same type */
p:nth-of-type(2) { font-style: italic; }

/* only-child / only-of-type */
p:only-child { text-align: center; }

/* :empty - elements with no children/content */
div:empty { display: none; }

/* :not() - exclude elements */
li:not(.active) { opacity: 0.6; }

/* :is() and :where() - group selectors concisely */
:is(h1, h2, h3) { font-family: "Georgia", serif; }
/* :where() has ZERO specificity, useful for overridable defaults */
:where(ul, ol) li { margin-bottom: 4px; }
```

```
<ul>
  <li>Item 1</li>
  <li class="active">Item 2</li>
  <li>Item 3</li>
</ul>
```

10.4 The `:has()` Relational Pseudo-class (Modern)

Selects an element IF it contains a certain descendant — a "parent selector" finally possible in pure CSS.

```
/* Style a card differently if it contains an image */
.card:has(img) {
  display: flex;
  gap: 1rem;
}

/* Highlight a form-group if its input is invalid */
.form-group:has(input:invalid) {
  border-left: 3px solid red;
}

/* Style a label based on a sibling checkbox state */
label:has(+ input:checked) {
  color: green;
}
```

10.5 Pseudo-elements

```
/* ::before and ::after: insert generated content (requires `content`) */
.quote::before {
  content: "\201C"; /* opening curly quote */
  font-size: 2em;
  color: #999;
}

.required::after {
  content: " *";
  color: red;
}

/* Common pattern: clearfix / decorative elements */
.divider::after {
  content: "";
  display: block;
  width: 50px;
  height: 3px;
  background: #2563eb;
  margin-top: 8px;
}

/* ::first-line and ::first-letter */
p::first-line {
  font-weight: bold;
}

p::first-letter {
  font-size: 3em;
  float: left;
  line-height: 1;
  padding-right: 4px;
}

/* ::placeholder - styling input placeholder text */
input::placeholder {
  color: #999;
  font-style: italic;
}

/* ::selection - styling user-highlighted text */
::selection {
  background: #ffe680;
  color: #000;
}

/* ::marker - styling list bullet/number */
li::marker {
  color: #2563eb;
  font-weight: bold;
}
```

MODULE 11: Transitions & Animations

11.1 CSS Transitions

Transitions smoothly animate a property from one value to another when a state changes (e.g., hover).

```
.button {
  background-color: #2563eb;
  color: white;
  padding: 0.75rem 1.5rem;
  border-radius: 6px;
  border: none;
  cursor: pointer;

  /* property | duration | easing | delay */
  transition: background-color 0.3s ease, transform 0.2s ease;
}

.button:hover {
  background-color: #1d4ed8;
  transform: translateY(-2px);
}

/* Transition ALL properties */
.card {
  transition: all 0.4s ease-in-out;
}

/* Timing functions */
.t1 { transition-timing-function: ease; }        /* slow-fast-slow (default) */
.t2 { transition-timing-function: linear; }       /* constant speed */
.t3 { transition-timing-function: ease-in; }      /* starts slow */
.t4 { transition-timing-function: ease-out; }     /* ends slow */
.t5 { transition-timing-function: ease-in-out; }  /* slow both ends */
.t6 { transition-timing-function: cubic-bezier(0.68,-0.55,0.27,1.55); } /* spring */
```

11.2 CSS Animations with `@keyframes`

Animations run independently without needing a trigger state. They use `@keyframes` to define stages.

```
/* Define the animation */
@keyframes fadeIn {
  from { opacity: 0; transform: translateY(20px); }
  to   { opacity: 1; transform: translateY(0); }
}

@keyframes pulse {
  0%   { transform: scale(1); }
  50%  { transform: scale(1.05); }
  100% { transform: scale(1); }
}

@keyframes slideRight {
  0%   { transform: translateX(-100%); }
  100% { transform: translateX(0); }
}

/* Apply animation */
.hero-text {
  animation-name: fadeIn;
  animation-duration: 0.6s;
  animation-timing-function: ease-out;
  animation-delay: 0.2s;
  animation-iteration-count: 1;     /* or: infinite */
  animation-direction: normal;      /* normal | reverse | alternate | alternate-reverse */
  animation-fill-mode: both;        /* none | forwards | backwards | both */
  animation-play-state: running;    /* running | paused */
}

/* Shorthand: name | duration | easing | delay | iterations | direction | fill-mode */
.hero-text {
  animation: fadeIn 0.6s ease-out 0.2s both;
}

/* Multiple animations */
.badge {
  animation: pulse 2s ease-in-out infinite, fadeIn 0.5s ease both;
}
```

11.3 The `transform` Property

Transform manipulates an element's position, rotation, size, and skew without affecting document flow.

```
.transform-examples {
  transform: translateX(50px);            /* move right 50px */
  transform: translateY(-20px);           /* move up 20px */
  transform: translate(50px, -20px);      /* move X and Y */
  transform: rotate(45deg);               /* rotate clockwise */
  transform: scale(1.5);                  /* 150% of original size */
  transform: scale(0.5, 1);               /* scale X and Y separately */
  transform: skewX(15deg);                /* skew along X axis */
  transform: skewY(5deg);                 /* skew along Y axis */

  /* Chain multiple transforms in ONE rule */
  transform: translateY(-5px) scale(1.02) rotate(3deg);
}

/* Transform origin: the pivot point for rotation/scale */
.logo {
  transform-origin: center center; /* default */
  transform-origin: top left;
  transform-origin: 50% 50%;
}
```

11.4 3D Transforms & Perspective

```css
.scene {
  perspective: 800px; /* sets depth for 3D children */
}

.card-flip {
  width: 200px;
  height: 300px;
  transform-style: preserve-3d;
  transition: transform 0.6s ease;
}

.card-flip:hover {
  transform: rotateY(180deg);
}

.card-front, .card-back {
  position: absolute;
  width: 100%;
  height: 100%;
  backface-visibility: hidden;
}

.card-back {
  transform: rotateY(180deg);
  background: #2563eb;
  color: white;
}
```

MODULE 12: CSS Custom Properties (Variables) & Advanced Selectors

12.1 CSS Custom Properties (Variables)

CSS variables are defined with `--` prefix, read with `var()`, and are scoped to the element they're defined on (and its children).

```
/* Define on :root to make globally available */
:root {
  --color-primary: #2563eb;
  --color-secondary: #7c3aed;
  --color-bg: #0f172a;
  --color-text: #e2e8f0;

  --spacing-sm: 0.5rem;
  --spacing-md: 1rem;
  --spacing-lg: 2rem;

  --border-radius: 8px;
  --font-sans: "Inter", sans-serif;
  --shadow: 0 4px 16px rgba(0, 0, 0, 0.25);
}

/* Use them */
.button {
  background: var(--color-primary);
  color: white;
  padding: var(--spacing-sm) var(--spacing-md);
  border-radius: var(--border-radius);
  font-family: var(--font-sans);
}

/* Fallback value if variable is not defined */
.box {
  color: var(--undefined-color, black);
}
```

12.2 Dynamic Theming with CSS Variables

CSS variables are reactive — changing them updates all elements that use them, great for dark mode.

```css
:root {
  --bg: #ffffff;
  --text: #111827;
  --card-bg: #f9fafb;
}

[data-theme="dark"] {
  --bg: #0f172a;
  --text: #e2e8f0;
  --card-bg: #1e293b;
}

body {
  background: var(--bg);
  color: var(--text);
}

.card {
  background: var(--card-bg);
}
```

```javascript
// Toggle dark mode via JavaScript
document.documentElement.setAttribute("data-theme", "dark");
```

12.3 CSS Variables in Calculations

```css
:root {
  --columns: 3;
  --gap: 1rem;
}

.grid-item {
  width: calc((100% - var(--gap) * (var(--columns) - 1)) / var(--columns));
}
```

12.4 Attribute Selectors

```css
/* Has attribute */
a[href]           { color: blue; }

/* Exact value */
input[type="text"] { border: 1px solid #ccc; }

/* Starts with */
a[href^="https"] { color: green; }

/* Ends with */
a[href$=".pdf"]::after { content: " (PDF)"; }

/* Contains */
a[href*="example"] { font-weight: bold; }

/* Space-separated word match */
[class~="active"] { outline: 2px solid blue; }

/* Dash-separated prefix (good for lang attributes) */
[lang|="en"] { font-family: "Times New Roman"; }
```

12.5 The `:is()`, `:where()`, `:not()`, `:has()` Power Combo

```
/* :is() - matches any in the list, takes HIGHEST specificity among them */
:is(h1, h2, h3, h4) {
  line-height: 1.2;
}

/* :where() - same but ZERO specificity — easy to override */
:where(p, ul, ol) {
  margin-bottom: 1rem;
}

/* :not() with complex selectors */
.nav-link:not(.active):not(:hover) {
  color: #94a3b8;
}

/* :has() as parent selector */
.form-group:has(> input:placeholder-shown) label {
  transform: translateY(0);
}

.form-group:has(> input:not(:placeholder-shown)) label {
  transform: translateY(-24px) scale(0.85);
}
```

12.6 `@layer` — Cascade Layers (Modern)

Cascade layers give you explicit control over specificity and ordering of your CSS without fighting specificity wars.

```
/* Declare layers in order of priority (last = highest) */
@layer reset, defaults, components, utilities;

@layer reset {
  * { margin: 0; padding: 0; box-sizing: border-box; }
}

@layer defaults {
  body { font-family: sans-serif; line-height: 1.5; }
  a { color: inherit; }
}

@layer components {
  .button {
    background: var(--color-primary);
    padding: 0.5rem 1rem;
    border-radius: 4px;
  }
}

@layer utilities {
  .text-center { text-align: center; }
  .hidden { display: none; }
}
```

MODULE 13: Advanced Visual Effects

13.1 Filters

The `filter` property applies visual effects similar to Photoshop adjustments.

```
.filter-demos {
  filter: blur(4px);
  filter: brightness(1.4);       /* > 1 = brighter */
  filter: contrast(200%);
  filter: grayscale(100%);       /* 0–100% */
  filter: hue-rotate(90deg);
  filter: invert(100%);
  filter: opacity(0.5);
  filter: saturate(2);
  filter: sepia(100%);
  filter: drop-shadow(4px 4px 8px rgba(0,0,0,0.5));

  /* Chain multiple filters */
  filter: brightness(1.1) contrast(1.1) saturate(1.2);
}

/* Hover image effect */
.hero-image {
  filter: grayscale(100%);
  transition: filter 0.4s ease;
}

.hero-image:hover {
  filter: grayscale(0%);
}
```

13.2 `backdrop-filter` — Glassmorphism Effect

Applies filters to the content BEHIND an element (requires a semi-transparent background).

```
.glass-card {
  background: rgba(255, 255, 255, 0.08);
  backdrop-filter: blur(12px) saturate(180%);
  -webkit-backdrop-filter: blur(12px) saturate(180%); /* Safari */
  border: 1px solid rgba(255, 255, 255, 0.12);
  border-radius: 16px;
  box-shadow: 0 8px 32px rgba(0, 0, 0, 0.3);
  color: white;
  padding: 2rem;
}
```

13.3 `clip-path` — Shape Clipping

Clip elements to custom shapes.

```
.clip-examples {
  /* Polygon points as percentages */
  clip-path: polygon(50% 0%, 100% 100%, 0% 100%);  /* triangle */
  clip-path: polygon(0 0, 100% 0, 100% 85%, 0 100%); /* diagonal bottom */
  clip-path: circle(50% at 50% 50%);               /* circle */
  clip-path: ellipse(60% 40% at 50% 50%);
  clip-path: inset(10px 20px round 8px);            /* inset rectangle */
}

/* Animated clip-path */
.wipe-reveal {
  clip-path: polygon(0 0, 0 0, 0 100%, 0 100%);
  transition: clip-path 0.6s ease;
}

.wipe-reveal.active {
  clip-path: polygon(0 0, 100% 0, 100% 100%, 0 100%);
}
```

13.4 `mix-blend-mode` & `background-blend-mode`

Control how an element blends with what's behind it (like Photoshop layer blend modes).

```
.blend-overlay {
  mix-blend-mode: overlay;
  mix-blend-mode: multiply;
  mix-blend-mode: screen;
  mix-blend-mode: difference;   /* great for dark/light theme text effects */
  mix-blend-mode: luminosity;
}

.image-with-color-overlay {
  position: relative;
}

.image-with-color-overlay::after {
  content: "";
  position: absolute;
  inset: 0; /* shorthand for top/right/bottom/left: 0 */
  background: rgba(100, 0, 255, 0.4);
  mix-blend-mode: multiply;
}
```

13.5 `mask` and `mask-image`

Masks hide parts of an element using gradients or images (like a non-destructive crop).

```css
/* Fade bottom with gradient mask */
.fade-bottom {
  mask-image: linear-gradient(to bottom, black 70%, transparent 100%);
  -webkit-mask-image: linear-gradient(to bottom, black 70%, transparent 100%);
}

/* Circular mask */
.avatar {
  mask-image: radial-gradient(circle, black 50%, transparent 51%);
}

/* Image-based mask (transparent = show, opaque = hide) */
.fancy-mask {
  mask-image: url("mask-shape.svg");
  mask-size: cover;
  mask-repeat: no-repeat;
}
```

13.6 Scroll Snap

Snap scroll position to specific points — useful for carousels and full-page sections.

```css
.scroll-container {
  scroll-snap-type: x mandatory; /* axis | none/mandatory/proximity */
  overflow-x: scroll;
  display: flex;
}

.scroll-item {
  scroll-snap-align: start; /* start | center | end */
  flex: 0 0 100%;
}
```

```html
<div class="scroll-container">
  <section class="scroll-item">Slide 1</section>
  <section class="scroll-item">Slide 2</section>
  <section class="scroll-item">Slide 3</section>
</div>
```

13.7 `aspect-ratio`

Maintain a specific width:height ratio without padding hacks.

```css
.video-embed {
  width: 100%;
  aspect-ratio: 16 / 9;
}

.square-avatar {
  width: 64px;
  aspect-ratio: 1 / 1;
  border-radius: 50%;
  object-fit: cover;
}

.golden-ratio-box {
  width: 100%;
  aspect-ratio: 1.618 / 1;
}
```

MODULE 14: CSS Architecture & Best Practices

14.1 CSS Methodologies

BEM (Block Element Modifier) — Most Popular

BEM creates predictable, low-specificity class names that scale well.

```
/* Block — standalone component */
.card { }

/* Element — part of the block (double underscore) */
.card__title { }
.card__image { }
.card__body { }

/* Modifier — variation of block or element (double dash) */
.card--featured { }
.card--dark { }
.card__title--large { }
```

```
<article class="card card--featured">
  <img class="card__image" src="..." alt="...">
  <h2 class="card__title card__title--large">Title</h2>
  <div class="card__body">Body text here</div>
</article>
```

Utility-First (Tailwind-inspired pattern)

Small, single-purpose classes composed together directly in HTML.

```
.flex       { display: flex; }
.items-center { align-items: center; }
.gap-4      { gap: 1rem; }
.bg-blue-600 { background-color: #2563eb; }
.text-white { color: #fff; }
.p-4        { padding: 1rem; }
.rounded-lg { border-radius: 0.5rem; }
```

```
<div class="flex items-center gap-4 bg-blue-600 text-white p-4 rounded-lg">
  ...
</div>
```

14.2 CSS Custom Properties Design System Pattern

```
:root {
  /* Color Palette */
  --clr-primary-50:  #eff6ff;
  --clr-primary-500: #3b82f6;
  --clr-primary-900: #1e3a8a;

  /* Typography Scale */
  --text-xs:   0.75rem;
  --text-sm:   0.875rem;
  --text-base: 1rem;
  --text-lg:   1.125rem;
  --text-xl:   1.25rem;
  --text-2xl:  1.5rem;
  --text-4xl:  2.25rem;

  /* Spacing Scale */
  --space-1: 0.25rem;
  --space-2: 0.5rem;
  --space-4: 1rem;
  --space-8: 2rem;

  /* Component Tokens */
  --btn-bg: var(--clr-primary-500);
  --btn-radius: 6px;
  --btn-padding: var(--space-2) var(--space-4);
}
```

14.3 The CSS Reset / Normalize

Start with a clean slate before writing styles:

```
/* Modern minimal reset */
*, *::before, *::after {
  box-sizing: border-box;
  margin: 0;
  padding: 0;
}

html {
  font-size: 16px;
  -webkit-text-size-adjust: 100%;
}

body {
  line-height: 1.5;
  -webkit-font-smoothing: antialiased;
}

img, picture, video, canvas, svg {
  display: block;
  max-width: 100%;
}

input, button, textarea, select {
  font: inherit;
}

p, h1, h2, h3, h4, h5, h6 {
  overflow-wrap: break-word;
}
```

14.4 Responsive Design Patterns

```
/* The "Holy Grail" Layout — Sidebar + Main + optional second sidebar */
.holy-grail {
  display: grid;
  grid-template-areas:
    "header"
    "main"
    "sidebar"
    "footer";
  grid-template-rows: auto 1fr auto auto;
}

@media (min-width: 768px) {
  .holy-grail {
    grid-template-areas:
      "header  header"
      "sidebar main"
      "footer  footer";
    grid-template-columns: 250px 1fr;
    grid-template-rows: auto 1fr auto;
  }
}

/* Sidebar flips below content on mobile */
```

14.5 Performance Best Practices

```
/* Use transform & opacity for animations — GPU-composited, no reflow */
.animate-good {
  transition: transform 0.3s ease, opacity 0.3s ease; /* GOOD */
}

.animate-bad {
  transition: width 0.3s ease, left 0.3s ease; /* BAD: causes layout reflow */
}

/* will-change hints: tell browser to prep in advance */
.heavy-animation {
  will-change: transform; /* use sparingly, costs memory */
}

/* Contain layout calculation to a subtree */
.isolated-component {
  contain: layout style paint; /* browser can skip this subtree in global layout */
}

/* content-visibility: skip rendering off-screen sections */
.below-fold-section {
  content-visibility: auto;
  contain-intrinsic-size: 0 500px;
}
```

14.6 Accessibility in CSS

```css
/* Never remove outlines without replacing them */
:focus-visible {
  outline: 2px solid #2563eb;
  outline-offset: 4px;
}

/* Visually hidden but accessible to screen readers */
.sr-only {
  position: absolute;
  width: 1px;
  height: 1px;
  padding: 0;
  margin: -1px;
  overflow: hidden;
  clip: rect(0, 0, 0, 0);
  white-space: nowrap;
  border: 0;
}

/* Respect user motion preferences */
@media (prefers-reduced-motion: reduce) {
  *, *::before, *::after {
    animation-duration: 0.01ms !important;
    transition-duration: 0.01ms !important;
  }
}

/* High contrast support */
@media (prefers-contrast: high) {
  .card {
    border: 2px solid currentColor;
  }
}

/* Minimum touch target size */
.button {
  min-width: 44px;
  min-height: 44px;
}
```

MODULE 15: Modern & Cutting-Edge CSS (2024–2026)

15.1 CSS Nesting (Native — No Sass Required)

CSS now supports nesting natively in all modern browsers.

```
.card {
  background: white;
  padding: 1.5rem;
  border-radius: 8px;

  /* Nested element selectors */
  .card-title {
    font-size: 1.25rem;
    margin-bottom: 0.5rem;
  }

  .card-body {
    color: #666;
  }

  /* Nested pseudo-class (& = parent reference) */
  &:hover {
    box-shadow: 0 8px 24px rgba(0,0,0,0.15);
    transform: translateY(-2px);
  }

  /* Nested media query */
  @media (min-width: 768px) {
    padding: 2rem;
  }

  /* Combinators in nesting */
  & + & {
    margin-top: 1rem;
  }
}
```

15.2 `@scope` — Scoped Styles

Apply styles only within a particular part of the DOM without high specificity.

```
@scope (.card) {
  :scope {
    border: 1px solid #eee;
  }

  .title {
    font-weight: 700;
  }

  .body {
    color: #666;
  }
}
```

15.3 Color Functions & Color Spaces (Modern)

```
:root {
  /* oklch: perceptually uniform, great for accessible palettes */
  --primary: oklch(55% 0.2 250);

  /* lch, lab for wide-gamut displays */
  --accent: lch(60% 80 200);
}

/* color-mix(): blend two colors */
.mixed {
  background: color-mix(in srgb, blue 30%, white);
  color: color-mix(in oklch, var(--primary) 50%, black);
}

/* relative color syntax */
.lighter-primary {
  color: oklch(from var(--primary) calc(l + 0.2) c h);
}
```

15.4 `@starting-style` — Entry Animations Without JavaScript

Animate elements when they first appear (e.g., dialog, popover) using only CSS.

```
dialog {
  opacity: 1;
  transform: translateY(0);
  transition: opacity 0.3s ease, transform 0.3s ease,
              overlay 0.3s ease allow-discrete,
              display 0.3s ease allow-discrete;
}

/* Styles before the element appears */
@starting-style {
  dialog[open] {
    opacity: 0;
    transform: translateY(-20px);
  }
}
```

15.5 CSS `@property` — Typed Custom Properties

Register CSS variables with a type, syntax, and inheritance, enabling animation of normally non-animatable values.

```
@property --gradient-angle {
  syntax: "<angle>";
  inherits: false;
  initial-value: 0deg;
}

.animated-gradient {
  background: linear-gradient(var(--gradient-angle), #ff7e5f, #feb47b);
  animation: rotate-gradient 3s linear infinite;
}

@keyframes rotate-gradient {
  to { --gradient-angle: 360deg; }
}
```

15.6 Popover API Styling (HTML + CSS)

The native HTML Popover API needs minimal CSS.

```
/* Style the backdrop behind a popover */
.modal::backdrop {
  background: rgba(0,0,0,0.5);
  backdrop-filter: blur(4px);
}

[popover] {
  border: none;
  border-radius: 12px;
  padding: 2rem;
  box-shadow: 0 20px 60px rgba(0,0,0,0.3);
}
```

```
<button popovertarget="my-popover">Open</button>
<div id="my-popover" popover class="modal">
  <h2>Hello from a native popover!</h2>
</div>
```

15.7 `text-wrap: balance` and `text-wrap: pretty`

Fix typographic widows and ragged lines automatically.

```
h1, h2, h3 {
  text-wrap: balance; /* evenly wraps heading text across lines */
}

p {
  text-wrap: pretty; /* avoids single word on last line (widows) */
}
```

15.8 Logical Properties

Write layout-independent styles that adapt to different writing modes (LTR, RTL, vertical text).

```
.box {
  /* Logical equivalents of physical properties */
  margin-inline: auto;          /* margin-left + margin-right */
  padding-block: 1rem;          /* padding-top + padding-bottom */
  border-inline-start: 4px solid blue; /* left in LTR, right in RTL */
  inset-inline-start: 0;        /* left in LTR */
  inline-size: 100%;            /* width */
  block-size: 200px;            /* height */
}
```

15.9 `field-sizing: content` — Auto-Sizing Form Inputs

Make textarea/input grow with content with one CSS line.

```
textarea {
  field-sizing: content;
  min-block-size: 3lh;  /* minimum 3 lines tall */
  max-block-size: 10lh;
}

input[type="text"] {
  field-sizing: content;
  min-inline-size: 10ch;
}
```

15.10 CSS Anchor Positioning

Position elements relative to another element's anchor point — great for tooltips without JavaScript.

```
.anchor-button {
  anchor-name: --my-anchor;
}

.tooltip {
  position: absolute;
  position-anchor: --my-anchor;
  bottom: anchor(top);
  left: anchor(center);
  translate: -50% 0;
}
```

Quick Reference Card

Specificity Calculator

Selector	Score
Inline style	1000
`#id`	100
`.class`, `[attr]`, `:pseudo-class`	10
`element`, `::pseudo-element`	1

Essential `display` Values

Value	Behaviour
`block`	Full width, new line, respects all margin/padding
`inline`	Content width, no new line, ignores width/height
`inline-block`	Flows inline, respects all box model
`flex`	Flexbox container
`grid`	Grid container
`none`	Removes from layout

Flexbox Quick Reference

Container Property	Purpose
`justify-content`	Main axis alignment
`align-items`	Cross axis alignment
`gap`	Space between items
`flex-wrap`	Allow wrapping

Grid Quick Reference

Property	Purpose
`grid-template-columns`	Define columns
`grid-template-areas`	Named layout regions
`repeat(auto-fit, minmax())`	Responsive columns
`grid-column: 1 / -1`	Span full row

Units Cheat Sheet

Unit	Relative To
`px`	Screen pixel (absolute)
`rem`	Root `<html>` font-size
`em`	Parent font-size
`%`	Parent element size
`vw` / `vh`	Viewport width/height
`ch`	Width of "0" character
`fr`	Available grid space
`clamp(min, val, max)`	Responsive range

End of Complete CSS Mastery Guide — All 15 Modules